

Beginners Guide to Regression Analysis and Visualize Interpretations

Table Of Contents:

- **Understanding Regression**
- **How Does Regression Work?**
- **Types of Algorithms**
- **Testing of Algorithms**

Algorithms that we will consider:-

1. Simple Linear Regression
2. Multiple Linear Regression
3. Polynomial Regression

What is Regression?

Regression is a technique used to predict value of one variable(Dependent Variable) on the basis of other variables(Independent Variables). It is parametric in nature because it makes certain assumptions based on the data set. If the data set follows those assumptions, regression gives incredible results. Otherwise, it struggles to provide convincing accuracy.

How Does Regression Work?

Regression is a part of supervised learning which basically means that we train our models on the basis of given training data and our model tries to relate between the dependent and the independent variable. It does this using various functions that maps the independent variables

to the dependent variables. When the model is completely trained and the error is minimised then we are able to make predictions on testing data as well.

The 7 Steps of Machine Learning

1 Data Collection

- The quantity & quality of your data dictate how accurate our model is
- The outcome of this step is generally a representation of data which we will use for training
- Using pre-collected data, by way of datasets from Kaggle, UCI, etc., still fits into this step

2 Data Preparation

- Wrangle data and prepare it for training
- Clean that which may require it (remove duplicates, correct errors, deal with missing values, normalization, data type conversions, etc.)
- Randomize data, which erases the effects of the particular order in which we collected and/or otherwise prepared our data
- Visualize data to help detect relevant relationships between variables or class imbalances (bias alert!), or perform other exploratory analysis
- Split into training and evaluation sets (Good train/eval split? 80/20, 70/30, or similar, depending on domain, data availability, dataset particulars, etc.)

3 Choose a Model

- Different algorithms are for different tasks; choose the right one

4 Train the Model

- The goal of training is to answer a question or make a prediction correctly as often as possible
- Linear regression example: algorithm would need to learn values for m (or W) and b (x is input, y is output)
- Each iteration of process is a training step

5 Evaluate the Model

- Uses some metric or combination of metrics to "measure" objective performance of model
- Test the model against previously unseen data
- This unseen data is meant to be somewhat representative of model performance in the real world, but still helps tune the model (as opposed to test data, which does not)

6 Parameter Tuning

- This step refers to hyperparameter tuning, which is an "artform" as opposed to a science
- Tune model parameters for improved performance
- Simple model hyperparameters may include: number of training steps, learning rate, initialization values and distribution, etc.

7 Make Predictions

- Using further (test set) data which have, until this point, been withheld from the model (and for which class labels are known), are used to test the model; a better approximation of how the model will perform in the real world

Simple Linear Regression

It is a basic and commonly used type of predictive analysis. These regression estimates are used to explain the relationship between one dependent variable and one or more independent variables.

$$y = \beta + \alpha * X$$

where:

- y – Dependent Variable
- X – Independent variable
- β – intercept
- α – Slope

Data Collection & Data Preparation:

In this section, you may have to collect data sets from different sources, from the internet or your personal repository. You also need to do data cleaning if necessary, then separate training data and testing data. Here, I already have a separate data set, so no separation of data is required.

```
# Importing Libraries
import numpy as np                # working with array
import pandas as pd              # import data set
import matplotlib.pyplot as plt  # for visualization

# Import Training Data Set
Training_Dataset = pd.read_csv("train.csv")
Training_Dataset = Training_Dataset.dropna()
X_train = np.array(Training_Dataset.iloc[:, :-1].values) # Independent Variable
y_train = np.array(Training_Dataset.iloc[:, 1].values)   # Dependent Variable

# Import Testing Data Set
Testing_Dataset = pd.read_csv("test.csv")
Testing_Dataset = Testing_Dataset.dropna()
X_test = np.array(Testing_Dataset.iloc[:, :-1].values)  # Independent Variable
y_test = np.array(Testing_Dataset.iloc[:, 1].values)    # Dependent Variable
```

Training the Model

In this section, the selection of the best regression model is carried out using the `LinearRegression` library.

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)

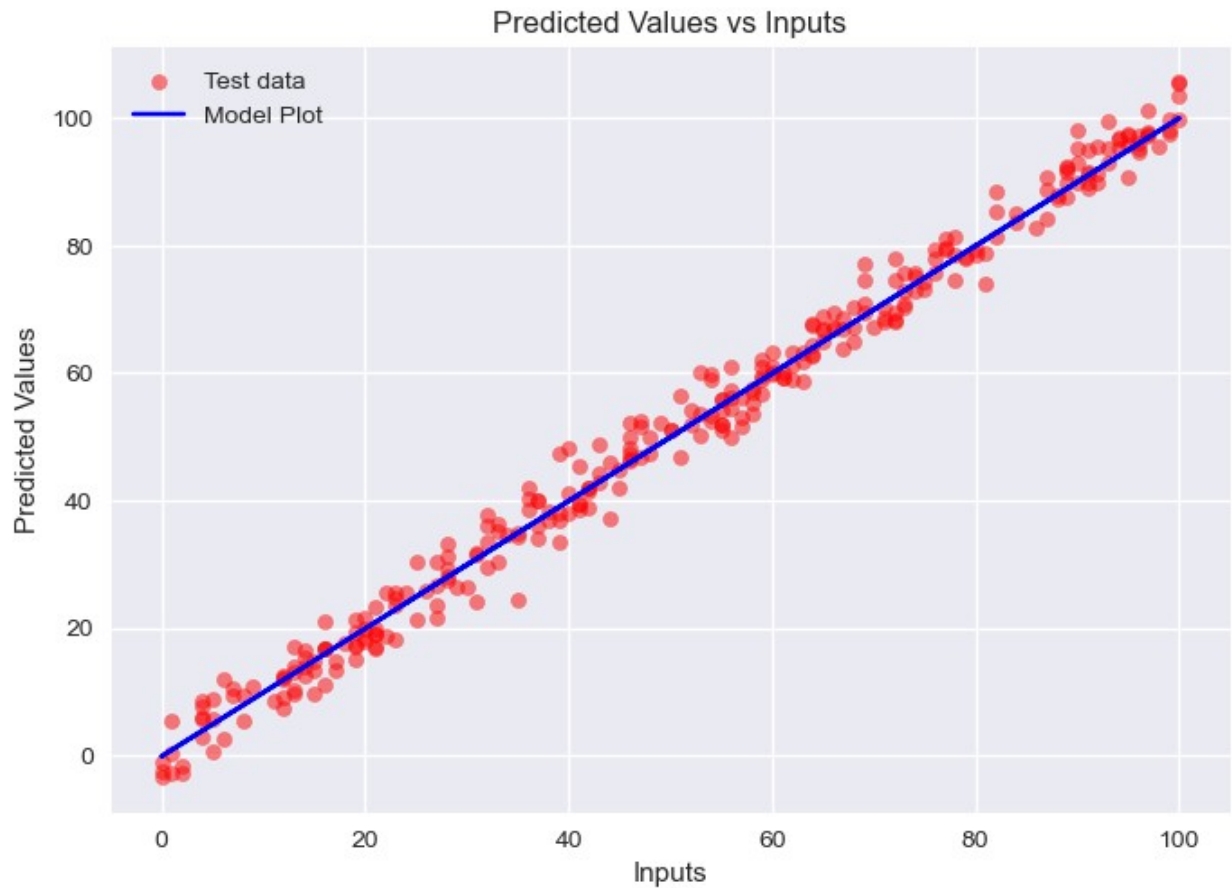
LinearRegression()

accuracy = regressor.score(X_test, y_test)
print('Accuracy = ' + str(accuracy))

Accuracy = 0.9888014444327563

plt.style.use('seaborn')
plt.scatter(X_test, y_test, color = 'red', marker = 'o', s = 35, alpha
= 0.5,
            label = 'Test data')
plt.plot(X_train, regressor.predict(X_train), color = 'blue',
label='Model Plot')
plt.title('Predicted Values vs Inputs')
plt.xlabel('Inputs')
plt.ylabel('Predicted Values')
plt.legend(loc = 'upper left')
plt.show()

C:\Users\Silicon computers\AppData\Local\Temp\
ipykernel_9960\3251397070.py:1: MatplotlibDeprecationWarning: The
seaborn styles shipped by Matplotlib are deprecated since 3.6, as they
no longer correspond to the styles shipped by seaborn. However, they
will remain available as 'seaborn-v0_8-<style>'. Alternatively,
directly use the seaborn API instead.
    plt.style.use('seaborn')
```



Multiple Linear Regression

It is also a basic and commonly used type of predictive analysis. These regression estimates are used to explain the relationship between one dependent variable and one or more independent variables.

$$Y = \beta + \alpha_1 * X_1 + \alpha_2 * X_2 + \dots + \alpha_n * X_n$$

where:

- Y : Dependent Variable
- X_1, X_2, X_3, X_4 : Independent variable
- β : Intercept
- $\alpha_1, \alpha_2, \dots, \alpha_n$: Slopes

Preparing Data:

```
dataset = pd.read_csv('insurance.csv')
dataset
```

	age	sex	bmi	children	smoker	region	expenses
0	19	female	27.9	0	yes	southwest	16884.92
1	18	male	33.8	1	no	southeast	1725.55
2	28	male	33.0	3	no	southeast	4449.46
3	33	male	22.7	0	no	northwest	21984.47
4	32	male	28.9	0	no	northwest	3866.86
...	...	...	...	...	...	...	...
1333	50	male	31.0	3	no	northwest	10600.55
1334	18	female	31.9	0	no	northeast	2205.98
1335	18	female	36.9	0	no	southeast	1629.83
1336	21	female	25.8	0	no	southwest	2007.95
1337	61	female	29.1	0	yes	northwest	29141.36

```
[1338 rows x 7 columns]
```

```
X = dataset.iloc[:, :-1] # Independent Variable
y = dataset.iloc[:, -1]  # Dependent Variable
```

```
# We have to apply encoding in the dataset as there are words present.
# for 'sex' and 'smoker' column we will apply Label Encoding as there
# are only 2 categories
# for 'region' we will apply OneHot Encoding as there are more than 2
# categories
```

```
# Label Encoding:
```

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
X.iloc[:, 1] = le.fit_transform(X.iloc[:, 1])
X.iloc[:, 4] = le.fit_transform(X.iloc[:, 4])
```

```
# OneHot Encoding:
```

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
ct = ColumnTransformer(transformers=[('encoder', OneHotEncoder(),
[5])], remainder='passthrough')
X = np.array(ct.fit_transform(X))
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
0.2, random_state = 0)
```

```
# Training the Model
```

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

```

LinearRegression()

accuracy = regressor.score(X_test, y_test)
print('Accuracy = ' + str(accuracy))

Accuracy = 0.8000184017333829

```

Polynomial Regression

It is a basic and commonly used type of predictive analysis. These regression estimates are used to explain the relationship between one dependent variable and one or more independent variables.

$$y = \beta + \alpha_1 * X + \alpha_2 * X^2 + \dots + \alpha_n * X^n$$

where:

- y : Dependent Variable
- $X, X^2, \dots, X^n$: Independent variable
- β : Intercept
- $\alpha_1, \alpha_2, \alpha_n$: Coefficients of independent variable

```
dataset = pd.read_csv('polynomial-regression.csv')
```

```

X = dataset.iloc[:, :-1] # Independent Variable
y = dataset.iloc[:, -1] # Dependent Variable
dataset

```

	araba_fiyat	araba_max_hiz
0	60	180
1	70	180
2	80	200
3	100	200
4	120	200
5	150	220
6	200	240
7	250	240
8	300	300
9	400	350
10	500	350
11	750	360
12	1000	365
13	2000	365
14	3000	365

```

# Trianing the Model
from sklearn.preprocessing import PolynomialFeatures
poly_reg = PolynomialFeatures(degree = 5)
X_poly = poly_reg.fit_transform(X)

```

```

lin_reg_2 = LinearRegression()
lin_reg_2.fit(X_poly, y)

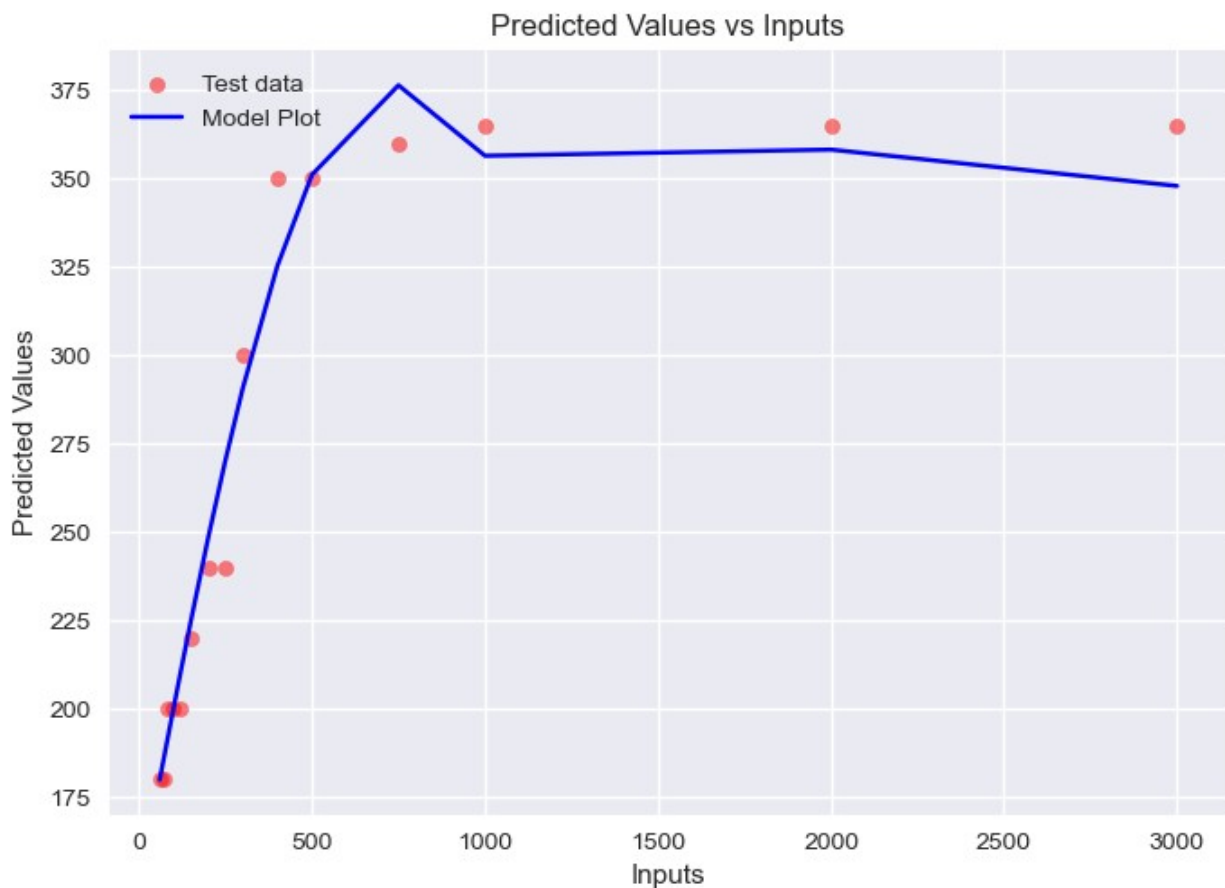
LinearRegression()

plt.style.use('seaborn')
plt.scatter(X, y, color = 'red', marker = 'o', s = 35, alpha = 0.5,
            label = 'Test data')
plt.plot(X, lin_reg_2.predict(poly_reg.fit_transform(X)), color =
'blue', label='Model Plot')
plt.title('Predicted Values vs Inputs')
plt.xlabel('Inputs')
plt.ylabel('Predicted Values')
plt.legend(loc = 'upper left')
plt.show()

```

C:\Users\Silicon computers\AppData\Local\Temp\ipykernel_9960\3950394777.py:1: MatplotlibDeprecationWarning: The seaborn styles shipped by Matplotlib are deprecated since 3.6, as they no longer correspond to the styles shipped by seaborn. However, they will remain available as 'seaborn-v0_8-*<style>*'. Alternatively, directly use the seaborn API instead.

```
plt.style.use('seaborn')
```



Thankyou: